

پروژه دوم سویچ p4 درس شبکه‌های کامپیوتری

ابزارها و پیش‌نیازها

- ماشین مجازی P4
 - Mininet
 - BMv2
 - Python
 - مثال آماده simple_int
-

ساختار پروژه

بخش اول: اجرای نمونه پایه INT (مربوط به دانشگاه ETH)

آدرس گیت هاب: <https://github.com/nsg-ethz/p4-learning.git>

هدف بخش

هدف این بخش آشنایی دانشجویان با:

- نحوه اجرای یک شبکه P4 در ماشین مجازی P4
- مفهوم In-Band Network Telemetry (INT)
- مشاهده عملی درج اطلاعات شبکه در داخل بسته‌ها

در این بخش هیچ تغییری در کد انجام نمی‌شود.

دریافت مخزن آموزشی P4

در ترمینال ماشین مجازی P4 دستور زیر را اجرا کنید:

```
cd ~git clone https://github.com/nsg-ethz/p4-learning.git
```

سپس وارد مسیر مثال شوید:

```
cd ~/p4-learning/examples/simple_int
```

اطمینان حاصل کنید که فایل زیر وجود دارد:

p4app.json

اجرای شبکه با استفاده از p4run

این مثال فاقد فایل **run.sh** است و باید با ابزار **p4run** اجرا شود.

دستور صحیح اجرا:

sudo p4run

```
p4@p4: ~/p4-learning/examples/simple_int
p4@p4:~/p4-learning/examples/simple_int/p4src$ cd ~/p4-learning/examples/simple_int
p4@p4:~/p4-learning/examples/simple_int$ sudo p4run
Reading JSON configuration file...
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udbpwtest mnexec ivs ryu-m
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udbpwtest mnexec ivs ryu-m
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([_[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Terminal: lng down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
brctl show | awk 'FNR > 1 {print $1}'
Compiling P4 files...
/home/p4/p4-learning/examples/simple_int/p4src/simple_int.p4 compiled successfully.
P4 Files compiled!
Port mapping:
h1: 0:s1
h2: 0:s2
h3: 0:s3
h4: 0:s3
s1: 1:h1      2:s2    3:s3
s2: 1:h2      2:s1
s3: 1:h3      2:h4    3:s1
Creating network...
*** Creating network
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1 s2 s3
*** Adding routers:
```

نکته مهم:

- این دستور باید از پوشه‌ای اجرا شود که فایل **p4app.json** در آن قرار دارد
- اجرای **p4run** از داخل پوشه **p4src** باعث خطا می‌شود.

پس از اجرای موفق، محیط Mininet نمایش داده می‌شود:

```
mininet>
```

```
p4@p4: ~/p4-learning/examples/simple_int
By default pcap directory is "./pcap".
*** Starting CLI:
mininet> h1 ping h2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=62 time=1.66 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=62 time=1.51 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=62 time=1.43 ms
64 bytes from 10.0.2.2: icmp_seq=4 ttl=62 time=1.42 ms
64 bytes from 10.0.2.2: icmp_seq=5 ttl=62 time=1.40 ms
64 bytes from 10.0.2.2: icmp_seq=6 ttl=62 time=1.60 ms
64 bytes from 10.0.2.2: icmp_seq=7 ttl=62 time=1.44 ms
64 bytes from 10.0.2.2: icmp_seq=8 ttl=62 time=1.43 ms
64 bytes from 10.0.2.2: icmp_seq=9 ttl=62 time=1.42 ms
```

اجرای گیرنده (receive.py) INT

در این مثال، فایل استخراج اطلاعات INT با نام `receive.py` ارائه شده است
(`receiver.py`).

اجرای صحیح گیرنده:

در محیط Mininet دستور زیر را اجرا کنید:

```
mininet> h2 python3 receive.py
```

دلیل اجرا از داخل Mininet:

- این اسکریپت روی اینترفیس `h2-eth0` شنود می‌کند
- این اینترفیس فقط داخل namespace مربوط به میزبان `h2` وجود دارد

پس از اجرا باید پیام زیر نمایش داده شود:

```
sniffing on h2-eth0
```

```
link/ether 00:00:0a:00:02:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
mininet> h2 python3 receive.py
sniffing on h2-eth0
^Cmininet>
```

ارسال ترافیک INT فعال‌سازی نمونه

استفاده از دستور `ping` برای این مثال کافی نیست،
زیرا `ping` مبتنی بر ICMP است و اطلاعات INT در این مثال تنها در بسته‌های UDP درج می‌شوند.

برای ارسال بسته‌های INT، از اسکریپت `send.py` استفاده کنید.

دستور صحیح ارسال بسته:

```
mininet> h1 python3 send.py 10.0.2.2 "INT test" 5
```

```
^Cmininet>
mininet> h1 python3 send.py
pass 3 arguments: <destination> "<message>" <number of packets>
mininet> h1 python3 send.py 10.0.2.2 "INT test" 5
###[ Ethernet ]###
dst      = ff:ff:ff:ff:ff:ff
src      = 00:00:0a:00:01:01
type     = IPv4
###[ IP ]###
version  = 4
ihl      = 6
tos      = 0x0
len      = 40
id       = 1
flags    =
frag     = 0
ttl      = 64
proto    = udp
Show Applications 0x43be
src      = 10.0.1.1
dst      = 10.0.2.2
```

که در آن:

- 10.0.2.2 آدرس میزبان مقصد (h2) است
- "INT test" پیام دلخواه
- 5 تعداد بسته‌های ارسالی

مشاهده خروجی INT

پس از ارسال بسته‌ها، خروجی در ترمینالی که receive.py اجرا شده است نمایش داده می‌شود.

نمونه خروجی مورد انتظار:

```
--- INT packet received ---
Switch ID: 1
Queue Depth: 18
Output Port: 2

Switch ID: 2
Queue Depth: 24
Output Port: 1
```

این خروجی نشان می‌دهد که:

- هر سوئیچ در مسیر بسته
- اطلاعات محلی خود (شناسه، عمق صف و پورت خروجی)
- را به بسته اضافه کرده است

جمع‌بندی آموزشی این بخش

در این بخش، دانشجویان به‌صورت عملی مشاهده می‌کنند که:

- اطلاعات وضعیت شبکه می‌تواند درون خود بسته‌ها حمل شود
- نیازی به سیستم مانیتورینگ جداگانه وجود ندارد
- INT امکان مشاهده وضعیت لحظه‌ای شبکه در سطح hop-by-hop را فراهم می‌کند

خطاهای رایج (برای جلوگیری از سردرگمی)

- اجرای `p4run` از داخل پوشه `p4src`
- اجرای `receive.py` خارج از محیط Mininet
- استفاده از `ping` به‌جای `send.py`

خروجی مورد انتظار این بخش

- اسکرین شات از اجرای موفق شبکه
 - اسکرین شات از مشاهده اطلاعات INT در خروجی `receive.py`
 - درک عملی مفهوم In-Band Network Telemetry
-

بخش دوم پروژه:

هدف کلی

در این بخش، دانشجویان باید علاوه بر جمع‌آوری اطلاعات لحظه‌ای INT، یک تحلیل ساده از وضعیت شبکه انجام دهند. این تحلیل با الهام از چارچوب GEANT INT طراحی شده، اما به‌صورت ساده و قابل پیاده‌سازی در محیط آموزشی P4 و Mininet.

ایده اصلی (نسخه ساده‌شده)

در این بخش، دانشجویان باید نشان دهند که:

- می‌توان علاوه بر اطلاعات لحظه‌ای (Per-packet)
- آمار تجمعی (Aggregated Statistics) را نیز داخل سوئیچ P4 نگهداری کرد
- و سپس در سمت دریافت‌کننده، یک شاخص تحلیلی ساده از وضعیت شبکه استخراج نمود

وظایف دانشجویان – بخش سوئیچ (P4)

دانشجویان باید در برنامه P4 از همان فایلی که در بخش اول کلون کرده‌اند فایل `simple_int.p4` تغییرات زیر را اعمال کنند:

استفاده از Register در P4

در هر سوئیچ P4، با استفاده از `register` موارد زیر ذخیره شود:

- مجموع Queue Depth بسته‌های عبوری
- تعداد کل بسته‌های عبوری

نکته:

- Registerها باید در `Ingress pipeline` به‌روزرسانی شوند
- برای Queue Depth باید از:
- `standard_metadata.enq_qdepth`

استفاده شود

- توجه به **type casting** الزامی است (مثلاً تبدیل 19 بیت به 32 بیت)

حفظ INT لحظه‌ای (Per-hop)

همانند بخش‌های قبلی:

- Queue Depth لحظه‌ای
- Switch ID
- Output Port

باید همچنان در **INT header** هر بسته درج شود.

وظایف دانشجویان – بخش دریافت‌کننده (Python)

دانشجویان باید فایل `receive.py` را به‌گونه‌ای اصلاح کنند که:

اجرای receiver در محل صحیح

- برنامه `receive.py` نباید در شل اصلی لینوکس اجرا شود
- بلکه باید داخل **namespace** میزبان مقصد در **Mininet** اجرا شود، مثلاً:

```
mininet> h2 python3 receive.py
```

دلیل:

اینترفیس‌هایی مانند `h2-eth0` فقط داخل **namespace** مربوط به همان **host** وجود دارند.

استخراج اطلاعات INT

در `receive.py`، دانشجویان باید:

- **INT header** را از بسته IP استخراج کنند
- **Queue Depth** مربوط به هر **hop** را بخوانند
- اطلاعات هر سوئیچ را به‌صورت خوانا چاپ کنند **Switch ID**، **Queue Depth**، **Port**

محاسبه Average Queue Depth

در سمت دریافت‌کننده، برای هر بسته:

- Queue Depth تمام hop ها در مسیر جمع‌آوری شود
- Average Queue Depth مسیر محاسبه و چاپ شود:

$$\text{Average Queue Depth} = \frac{\sum \text{Queue Depths}}{\text{Number of Hops}}$$

تکرار آزمایش برای UDP و TCP

دانشجویان باید این آزمایش را برای هر دو نوع ترافیک انجام دهند:

حالت اول UDP :

- ارسال بسته‌ها با `send.py` پورت UDP مشخص
- دریافت و تحلیل INT در `receive.py`
- ثبت خروجی Average Queue Depth

حالت دوم TCP :

دانشجویان باید:

- ترافیک TCP مثلاً با `iperf` یک اسکریپت TCP ساده ایجاد کنند
- INT را برای بسته‌های TCP نیز فعال نگه دارند
- همان اطلاعات Queue Depth و Average را استخراج و گزارش کنند

توجه:

- نیازی به تغییر معماری INT برای TCP نیست
- فقط فیلتر دریافت در `Scapy` و نوع ترافیک متفاوت خواهد بود

خروجی مورد انتظار

دانشجویان باید در گزارش خود ارائه دهند:

- اسکرین شات خروجی نمونه INT برای UDP
- اسکرین شات خروجی نمونه INT برای TCP
- اسکرین شات مقدار Average Queue Depth برای هر حالت
- مقایسه‌ی کیفی:
 - UDP در برابر TCP

○ تفاوت رفتار صف‌ها
